

CER-Rec

反事实事件级语义 - 协同路由推荐

研究实施手册（SOP）

任务：LLM 增强的序列推荐 / Next-item Recommendation

目标：以最小可完成实验闭环，验证事件级路由优于用户级、物品级和请求级融合

版本 1.0 | 2026-07-01

执行摘要

本手册给出一套可以在有限算力与时间内完成、同时足够形成论文的完整实验方案。主线不是堆叠大量推荐模型，而是在统一的双专家架构中，比较用户级、物品级、用户-目标级和事件级路由，并进一步验证反事实贡献监督是否优于只依靠最终推荐损失的黑盒 Gate。

核心假设：同一用户历史内部，不同事件的协同证据与 LLM 语义证据具有不同可靠性。粗粒度 Gate 只能给整个用户、商品或请求分配一个比例，无法表达事件间差异。

最小可发表配置

模块	最终选择
数据集	Steam、Amazon Fashion、Amazon Beauty
评测	1 positive + 100 negatives; HR@10、NDCG@10; 3 seeds
协同专家	SASRec + DIN-style target-aware relevance
语义专家	Frozen Qwen3-Embedding-0.6B + Semantic SASRec + DIN-style relevance
外部 baseline	SASRec、LLMInit、RLMRec、LLM-ESR、HSUGA
受控 baseline	StaticFusion、UserGate、ItemGate、UserTargetGate、EventGate
完整方法	Keep Gate + Event Route Gate + Counterfactual Teacher
核心消融	w/o Keep、w/o Route、w/o Teacher、UserTarget-level
核心分析	Expert complementarity / Oracle; Noise history test

总执行顺序

```

数据预处理与固定负样本
↓
离线生成冻结的商品语义 Embedding
↓
独立训练 CF Expert 与 Semantic Expert
↓
Unique-hit / Oracle Go-No-Go 检查
↓
实现 UserGate / ItemGate / UserTargetGate / EventGate
↓
离线生成反事实事件贡献 Teacher
↓
训练 Keep Gate + Route Gate + Late Fusion Scorer
↓
补外部 baseline、消融、噪声实验、论文图表
  
```

目录

- 1. 研究问题与论文定位
- 2. 项目范围与优先级
- 3. 数据集与评测协议
- 4. 商品语义信息与 LLM 使用方式
- 5. 模型完整结构
- 6. Collaborative Expert
- 7. Semantic Expert
- 8. Keep Gate 与 Route Gate
- 9. 最终聚合、打分与张量维度
- 10. 完整训练流程
- 11. 反事实 Teacher 生成细节
- 12. 外部 Baseline 与受控 Baseline
- 13. 主结果、消融与分析实验
- 14. 超参数与代码结构
- 15. 脚本执行顺序与验收标准
- 16. 实验记录规范与六周计划
- 17. 论文最终结构与立即行动清单

1. 研究问题与论文定位

给定用户 u 的历史序列 $H_u = (h_1, h_2, \dots, h_L)$ 以及当前候选商品 t ，模型需要对每一条历史事件 h_j 做出两个独立判断。

1.1 Keep：这条历史是否与当前候选相关？

$$q_{\{u,t,j\}} \in [0, 1]$$

- q 接近 1：保留该历史事件。
- q 接近 0：将该历史视为噪声或无关证据。

1.2 Route：如果相关，应当相信哪种信息？

$$g_{\{u,t,j\}} \in [0, 1]$$

- g 接近 1：更依赖 collaborative signal。
- g 接近 0：更依赖 LLM semantic signal。

1.3 论文贡献的准确表述

本文不能声称“首次同时使用语义与协同信号”，也不能只声称“首次使用动态 Gate”。更准确的定位是：现有用户级或物品级方法只在粗粒度上估计信号可靠性，而本文在 user-target-history-event 粒度上分别判断事件保留与信号路由，并用反事实边际贡献为 Gate 提供具有可靠性含义的监督。

论文主问题：同一用户历史内部，语义证据和协同证据是否在不同事件上具有异质可靠性？事件级路由能否比用户级、物品级和用户-目标级融合更准确、更稳健？

2. 项目范围与优先级

2.1 P0：必须完成

- Steam、Amazon Fashion、Amazon Beauty 三个数据集。
- SASRec 协同专家与冻结 LLM Embedding 的语义专家。
- UserGate、ItemGate、UserTargetGate、EventGate。
- EventGate + Counterfactual Teacher 完整方法。
- SASRec、LLMInit、RLMRec、LLM-ESR、HSUGA 外部 baseline。
- HR@10、NDCG@10，3 个随机种子。
- Expert unique-hit、Oracle upper bound。
- 四个核心消融与一个随机噪声历史实验。

2.2 P1：核心结果稳定后补

- 第二个 backbone，例如 BERT4Rec。
- 第二个语义 encoder，验证方法不依赖单一 LLM Embedding。
- Head / Mid / Tail 商品分桶与 Short / Medium / Long 用户分桶。
- 路由校准、推理延迟和参数量。

2.3 P2：时间充足再做

- 多种 semantic/collaborative corruption。
- Shapley Value 或复杂反事实归因。
- Full-catalog evaluation。
- 联合微调 LLM 或生成式 OneRec 版本。

范围控制：第一轮只做 P0。不要在核心现象尚未验证时投入 OneRec、全库召回、LLM 微调或复杂因果归因。

3. 数据集与评测协议

3.1 数据集

数据集	用途	开发优先级
Amazon Beauty	主开发集；文本丰富、长尾明显，适合快速调试	第 1
Steam	相对更长行为序列，验证事件筛选与路由	第 2
Amazon Fashion	短序列与稀疏场景，验证粗粒度先验是否足够	第 3

3.2 数据切分

对每个用户按照时间排序：

$$(i_1, i_2, \dots, i_{\{T-2\}}, i_{\{T-1\}}, i_T)$$

- i_T 作为测试目标。
- $i_{\{T-1\}}$ 作为验证目标。
- 更早的交互构成训练序列。
- 不能随机打散用户历史；协同共现、item popularity 和用户活跃度只能由训练数据计算。

3.3 历史长度

`max_history_length: 50`

不足 50 的序列左侧 Padding，超过 50 时保留最近 50 条。核心结果完成后，再补 L = 10、20、50 的长度分析；Steam 资源允许时再补 100。

3.4 评测协议

每个测试样本：
1 个真实目标商品
+ 100 个该用户未交互负商品

指标：HR@10、NDCG@10
随机种子：2024、2025、2026

三个数据集和所有模型必须共享完全相同的验证/测试负样本文件。最终结果报告 mean ± std。

4. 商品语义信息与 LLM 使用方式

4.1 商品文本构造

Amazon 商品文本统一使用：

```
Title: ...
Brand: ...
Category: ...
Description: ...
Features: ...
```

Steam 商品文本统一使用：

```
Game name: ...
Genres: ...
Tags: ...
Developer: ...
Publisher: ...
Description: ...
```

不要引入测试时期评论、未来销量、全量 popularity 等可能泄漏的信息。缺失字段保留为空字符串并记录缺失标记。

4.2 主设置：冻结预训练 Embedding 模型

主实验使用 Qwen3-Embedding-0.6B 或同等规模的开源文本 Embedding 模型。模型仅在离线阶段编码商品文本，推荐训练期间不运行 LLM。

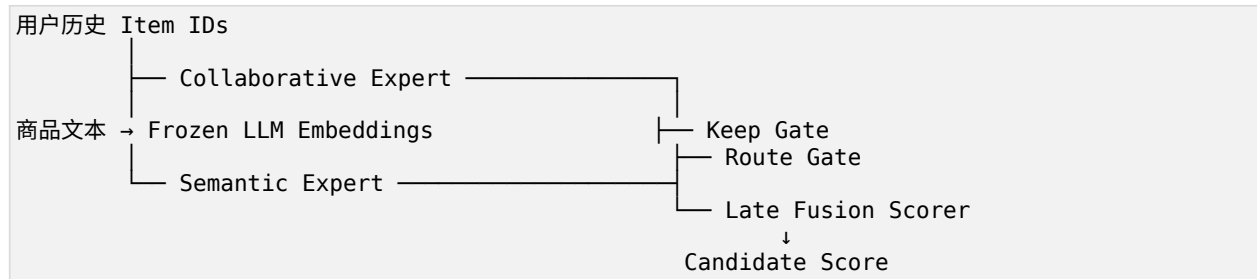
```
商品文本
→ Frozen Qwen3-Embedding-0.6B
→ 语义向量  $x_i^{sem}$  (例如 1024 维)
→ L2 Normalize / FP16 缓存
→ 可训练 Projector 映射至推荐 hidden_dim
```

$$e_i^{sem} = W_2 \cdot GELU(W_1 \cdot LN(x_i^{sem}))$$

4.3 训练与冻结边界

模块	是否训练
Qwen / 文本 Embedding 主干	冻结，离线编码
Semantic Projector	训练
Semantic SASRec	训练
Semantic target-aware relevance MLP	训练
CF SASRec 与 ID Embedding	训练
Keep / Route Gate	训练

5. 模型完整结构



5.1 两个专家必须保持独立

CF Expert 与 Semantic Expert 分别代表协同行为视角和语言语义视角。两者在预训练阶段参数不共享，避免两个分支学成相同表示，导致 disagreement 消失。

5.2 两个 Gate 的职责必须分离

Gate	问题	输出含义
Keep Gate q_j	历史事件是否值得保留?	0 = 过滤; 1 = 保留
Route Gate g_j	保留后更信 CF 还是 Semantic?	0 = Semantic; 1 = CF

6. Collaborative Expert

6.1 输入与序列编码

$$E^{cf} \in R^{|I| \times d}$$

历史 item ID Embedding 与位置 Embedding 输入 SASRec:

$$H^{cf} = SASRec_{cf}(e_{h_1}^{cf}, \dots, e_{h_L}^{cf})$$

$$H^{cf} = [v_1^{cf}, \dots, v_L^{cf}], \quad v_j^{cf} \in R^d$$

6.2 DIN-style Target-aware Relevance

当前候选商品 t 具有协同 Embedding e_t^{cf} 。对每条历史事件构造目标相关交叉特征:

$$x_j^{cf} = [v_j^{cf}; e_t^{cf}; v_j^{cf} \odot e_t^{cf}; v_j^{cf} - e_t^{cf}]$$

$$z_j^{cf} = MLP_{cf}(x_j^{cf})$$

$$r_j^{cf} = w_{cf}^T z_j^{cf}$$

$$\alpha_j^{cf} = softmax_j(r_j^{cf})$$

r_j^{cf} 表示: 从行为转移、共现和协同序列模式看, 历史事件 h_j 对候选 t 的支持程度。

7. Semantic Expert

7.1 输入与序列编码

$$e_i^{sem} = P_{sem}(x_i^{sem})$$

$$H^{\wedge\{sem\}} = SASRec_{\{sem\}}(e_{\{h_1\}^{\wedge\{sem\}}}, \dots, e_{\{h_L\}^{\wedge\{sem\}}})$$

$$H^{\wedge\{sem\}} = [v_{1^{\wedge\{sem\}}}, \dots, v_{L^{\wedge\{sem\}}}]$$

7.2 Target-aware Semantic Relevance

$$x_{j^{\wedge\{sem\}}} = [v_{j^{\wedge\{sem\}}}; e_{t^{\wedge\{sem\}}}; v_{j^{\wedge\{sem\}}} \odot e_{t^{\wedge\{sem\}}}; v_{j^{\wedge\{sem\}}} - e_{t^{\wedge\{sem\}}}]$$

$$z_{j^{\wedge\{sem\}}} = MLP_{\{sem\}}(x_{j^{\wedge\{sem\}}})$$

$$r_{j^{\wedge\{sem\}}} = w_{\{sem\}}^T z_{j^{\wedge\{sem\}}}$$

$$\alpha_{j^{\wedge\{sem\}}} = \text{softmax}_j(r_{j^{\wedge\{sem\}}})$$

$r_{j^{\wedge\{sem\}}}$ 表示：从标题、类别、描述等内容语义看，历史事件 h_j 对候选 t 的支持程度。

8. Keep Gate 与 Route Gate

8.1 Keep Gate

Keep Gate 的输入同时包含两位专家的事件表示、相关性分数和可解释统计信息：

$$x_{j^{\wedge\{keep\}}} = [z_{j^{\wedge\{cf\}}}; z_{j^{\wedge\{sem\}}}; r_{j^{\wedge\{cf\}}}; r_{j^{\wedge\{sem\}}}; |r_{j^{\wedge\{cf\}}} - r_{j^{\wedge\{sem\}}}|; m_u; m_t; m_{\{h_j\}}; \Delta time_j]$$

$$q_j = \text{sigmoid}(MLP_{\{keep\}}(x_{j^{\wedge\{keep\}}}))$$

- m_u ：用户历史长度或活跃度。
- m_t ：候选商品在训练集中的交互频率。
- $m_{\{h_j\}}$ ：历史商品在训练集中的交互频率。
- $\Delta time_j$ ：历史事件与当前目标之间的时间间隔。

8.2 Route Gate

$$x_{j^{\wedge\{route\}}} = [z_{j^{\wedge\{cf\}}}; z_{j^{\wedge\{sem\}}}; r_{j^{\wedge\{cf\}}}; r_{j^{\wedge\{sem\}}}; |r_{j^{\wedge\{cf\}}} - r_{j^{\wedge\{sem\}}}|; r_{j^{\wedge\{cf\}}} r_{j^{\wedge\{sem\}}}; m_u; m_t; m_{\{h_j\}}; \Delta time_j]$$

$$g_j = \text{sigmoid}(MLP_{\{route\}}(x_{j^{\wedge\{route\}}}))$$

- $g_j \rightarrow 1$ ：该事件主要使用协同证据。
- $g_j \rightarrow 0$ ：该事件主要使用语言语义证据。

重要设计：Keep 与 Route 不应由一个 Gate 同时承担。否则“无关事件”与“应该相信语义的事件”都可能产生较低 CF 权重，解释会混淆。

9. 最终聚合、打分与张量维度

9.1 Late Fusion 聚合

$$h_{\{u,t\}^{\wedge\{cf\}}} = \sum_j q_j g_j \alpha_{j^{\wedge\{cf\}}} v_{j^{\wedge\{cf\}}}$$

$$h_{\{u,t\}^{\wedge\{sem\}}} = \sum_j q_j (1 - g_j) \alpha_{j^{\wedge\{sem\}}} v_{j^{\wedge\{sem\}}}$$

语义与协同表示分别聚合，避免在早期强制对齐，保留两种视图各自的私有信息。

9.2 分支打分与跨视图打分

$$s_{\{cf\}}(u, t) = MLP_{\{cf-score\}}([h_{\{u, t\}^{\{cf\}}}; e_{t^{\{cf\}}}; h_{\{u, t\}^{\{cf\}}} \odot e_{t^{\{cf\}}})$$

$$s_{\{sem\}}(u, t) = MLP_{\{sem-score\}}([h_{\{u, t\}^{\{sem\}}}; e_{t^{\{sem\}}}; h_{\{u, t\}^{\{sem\}}} \odot e_{t^{\{sem\}}})$$

$$s_{\{cross\}}(u, t) = MLP_{\{cross\}}([h_{\{u, t\}^{\{cf\}}}; h_{\{u, t\}^{\{sem\}}}; h_{\{u, t\}^{\{cf\}}} \odot h_{\{u, t\}^{\{sem\}}}; h_{\{u, t\}^{\{cf\}}} \odot h_{\{u, t\}^{\{sem\}}})$$

$$s(u, t) = s_{\{cf\}}(u, t) + s_{\{sem\}}(u, t) + s_{\{cross\}}(u, t)$$

9.3 张量维度

```

B: batch size
L: history length
C: number of candidates
d: hidden dimension

CF history states:      [B, L, d]
Semantic history states: [B, L, d]
CF candidate embedding: [B, C, d]
Semantic candidate embedding: [B, C, d]
CF event features:      [B, C, L, 4d]
Semantic event features: [B, C, L, 4d]
Keep probabilities:      [B, C, L]
Route weights:           [B, C, L]
Final scores:            [B, C]
```

10. 完整训练流程

10.1 Stage 0: 数据与语义 Embedding

1. 下载或复用三个数据集的官方/公开预处理。
2. 生成 train、valid、test，并固定验证和测试负样本。
3. 构建商品文本并离线生成 semantic embedding。
4. 计算训练集 item frequency、user frequency 和时间间隔特征。

```

train.pkl
valid.pkl
test.pkl
valid_negatives.pkl
test_negatives.pkl
item_semantic_embeddings.npy
item_frequency.npy
user_frequency.npy
```

10.2 Stage 1: 独立训练 CF Expert

只启用 ID Embedding、CF SASRec、CF target-aware module 和 CF scoring head。

$$L_{cf} = -\log [\exp(s_{cf}(u, t+)) / (\exp(s_{cf}(u, t+)) + \sum_k \exp(s_{cf}(u, t-k)))]$$

- 训练负例：第一版使用 32 个随机负例。
- Event Dropout：以 10% 概率随机 Mask 历史事件，使专家适应后续反事实干预。
- 保存最佳验证 NDCG@10 checkpoint。

10.3 Stage 2：独立训练 Semantic Expert

只启用冻结语义 Embedding、Semantic Projector、Semantic SASRec、语义 target-aware module 和语义 scoring head。

$$L_{sem} = -\log [\exp(s_{sem}(u, t+)) / (\exp(s_{sem}(u, t+)) + \sum_k \exp(s_{sem}(u, t_k-)))]$$

- 同样使用 10% Event Dropout。
- Qwen 输出保持冻结；其余语义专家模块可训练。

10.4 Go / No-Go 检查

训练完两个专家后，必须先验证二者是否互补，再决定是否投入反事实 Teacher。

分组	定义
Both Correct	CF 与 Semantic 均在 Top-10 命中
CF-only Correct	仅 CF 命中
Semantic-only Correct	仅 Semantic 命中
Both Wrong	两者均未命中
Oracle	每个样本选择两专家中表现更好的一个

内部 Go 标准： Semantic-only 样本占比建议至少约 3%-5%，Oracle HR@10 相比最佳单专家建议至少高约 2 个百分点。该阈值仅用于内部决策，不是论文硬性标准。

10.5 Stage 3：先实现粗粒度与黑盒事件级 Gate

在生成 Teacher 前，先实现 StaticFusion、UserGate、ItemGate、UserTargetGate 和仅靠 L_rank 训练的 EventGate。这样可以先验证事件粒度本身是否有效。

10.6 Stage 4：训练反事实监督的完整模型

冻结两位专家，只训练 Keep Gate、Route Gate、分支 scorer 和 cross-view scorer。

$$L = L_{rank} + \lambda_k L_{keep} + \lambda_r L_{route}$$

第一版使用 Frozen Experts。核心结果稳定后，再补一次只解冻专家最后一层的小学习率联合微调。Qwen Embedding 始终冻结。

11. 反事实 Teacher 生成细节

11.1 选择需要干预的事件

候选事件集合 J =
 CF attention Top-10
 u Semantic attention Top-10
 u 随机采样 5 个低 attention 事件

Attention 仅用于减少 Teacher 生成成本，不直接作为可靠性标签。

11.2 排名 Margin

$$M_v(H) = s_v(u, t+) - \log \sum_k \exp(s_v(u, t_k-)), \quad v \in \{cf, sem\}$$

Teacher 阶段使用固定的 16 或 32 个负例，保证不同事件的贡献可比较。

11.3 Mask Intervention

$$H_{-j} = (h_1, \dots, [MASK]_j, \dots, h_L)$$

$$\Delta_j^v = M_v(H) - M_v(H_{-j})$$

- $\Delta > 0$: Mask 该事件后排名变差，该事件对专家有正贡献。
- $\Delta \approx 0$: 事件影响很小。
- $\Delta < 0$: 事件可能是噪声，移除后反而改善预测。

11.4 Keep Teacher

$$D_j = \max(0, \Delta_j^{cf}) + \max(0, \Delta_j^{sem})$$

$$y_j^{keep} = \text{sigmoid}((D_j - \tau_k) / T_k)$$

11.5 Route Teacher

$$y_j^{route} = \exp(\Delta_j^{cf} / T_r) / [\exp(\Delta_j^{cf} / T_r) + \exp(\Delta_j^{sem} / T_r)]$$

- 接近 1: CF 边际贡献更大。
- 接近 0: Semantic 边际贡献更大。
- 接近 0.5: 两边贡献相近。

11.6 置信权重

$$w_j = y_j^{keep} \cdot |\Delta_j^{cf} - \Delta_j^{sem}| / (|\Delta_j^{cf}| + |\Delta_j^{sem}| + \epsilon)$$

当两位专家贡献非常接近或事件本身无关时，Route Loss 权重较低，避免强行选边。

11.7 Router Loss

$$L_{keep} = \text{mean}_j \text{BCE}(q_j, y_j^{keep})$$

$$L_{route} = [\sum_j w_j \text{BCE}(g_j, y_j^{route})] / [\sum_j w_j + \epsilon]$$

$$L_{rank} = -\log [\exp(s(u, t+)) / (\exp(s(u, t+)) + \sum_k \exp(s(u, t_k-)))]$$

12. 外部 Baseline 与受控 Baseline

12.1 外部 Baseline

模型	代表范式	优先级
SASRec	纯协同序列推荐	必须
LLMInit	最简单的 LLM Embedding 注入	必须
RLMRec	语义与协同表示对齐	必须
LLM-ESR	双视图 LLM 语义与协同增强	最高
HSUGA	用户活跃度驱动的粗粒度语义利用	最高

优先使用官方代码和相同 processed data。不要一开始复现 GRU4Rec、BERT4Rec、PAD、FAERec、OneRec 等更多模型。

12.2 受控 Gate Baseline

所有受控 baseline 共享同一 CF Expert、Semantic Expert、语义 Embedding、hidden size、最终 scorer 和训练样本，只改变 Gate 粒度或监督。

模型	Gate 形式	验证问题
StaticFusion	$g = 0.5$	固定比例是否足够
UserGate	$g_{\{u,t,j\}} = g_u$	用户级先验是否足够
ItemGate	$g_{\{u,t,j\}} = g_t$	物品成熟度先验是否足够
UserTargetGate	$g_{\{u,t,j\}} = g_{\{u,t\}}$	请求级路由是否足够
EventGate	$g_{\{u,t,j\}}$, 仅 L_rank	事件粒度本身是否有效
Ours	Keep + EventGate + Teacher	反事实监督是否进一步有效

12.3 ItemGate 实现

$$g_t = \text{sigmoid}(\text{MLP}([e_t^{\{cf\}}; e_t^{\{sem\}}; \log(1+freq_t)]))$$

这是 GateSID-inspired 物品级路由，不应直接命名为完整 GateSID，因为没有复现其 Semantic ID 和工业 CTR 架构。

12.4 UserGate 实现

$$g_u = \text{sigmoid}(\text{MLP}([h_u^{\{cf\}}; h_u^{\{sem\}}; \log(1+H_u)]))$$

12.5 UserTargetGate 实现

$$g_{\{u,t\}} = \text{sigmoid}(\text{MLP}([h_u^{\{cf\}}; h_u^{\{sem\}}; e_t^{\{cf\}}; e_t^{\{sem\}}; m_u; m_t]))$$

13. 主结果、消融与分析实验

13.1 主结果表

SASRec
Semantic-only (建议保留)
LLMInit
RLMRec
LLM-ESR
HSUGA
Ours

三个数据集分别报告 HR@10 和 NDCG@10，3 个 seed 的 mean $\pm$ std。

13.2 Gate 粒度表

StaticFusion
UserGate
ItemGate
UserTargetGate
EventGate
EventGate + Counterfactual Teacher (Ours)

这张表是论文最核心的受控比较。预期总体趋势为：粗粒度 Gate < UserTargetGate < EventGate < EventGate + Teacher。

13.3 四个必须消融

消融	实现	验证内容
w/o Keep Gate	$q_j = 1$	历史筛选是否必要
w/o Route Gate	$g_j = 0.5$	信号选择是否必要
w/o Teacher	去掉 L_{keep} 与 L_{route}	反事实监督是否必要
UserTarget-level	所有事件共享 $g_{\{u,t\}}$	事件级粒度是否必要

13.4 Expert Complementarity / Oracle

在 Beauty 上至少报告 Both Correct、CF-only、Semantic-only、Both Wrong 和 Oracle。还可以计算两种 recovery:

$$CF-Recovery = |Ours\ Correct \cap CF-only| / |CF-only|$$

$$Sem-Recovery = |Ours\ Correct \cap Sem-only| / |Sem-only|$$

13.5 Noise History Test

向测试历史中注入 10 或 20 个随机无关商品, 比较 SASRec、LLM-ESR、HSUGA、ItemGate、EventGate 和 Ours。

$$\Delta HR@10 = HR@10_{clean} - HR@10_{noise}$$

第一轮只做随机噪声, 不需要立即扩展 semantic-hard 或 CF-hard distractor。

13.6 最低论文图表配置

编号	内容
Figure 1	动机: 同一历史中不同事件的 CF / Semantic 贡献差异
Figure 2	完整模型结构
Table 1	外部 Baseline 主结果
Table 2	Gate 粒度受控比较
Table 3	四个核心消融
Figure 3	Expert unique-hit 与 Oracle
Figure 4	Noise history 性能下降曲线
Figure 5	单个用户-候选案例的事件级 Keep / Route 可视化

14. 超参数与代码结构

14.1 起始超参数

```
hidden_dim: 64
max_history_length: 50
num_sasrec_layers: 2
num_attention_heads: 2
dropout: 0.2

expert_batch_size: 256
router_batch_size: 128
expert_lr: 1e-3
router_lr: 3e-4
joint_lr: 1e-4
weight_decay: 1e-5
```

```

train_negatives: 32
test_negatives: 100
event_dropout: 0.1

teacher_topk_cf: 10
teacher_topk_sem: 10
teacher_random_events: 5
lambda_keep: 0.3
lambda_route: 0.3
route_temperature: 0.5
keep_temperature: 0.5

early_stop_patience: 10
max_epochs: 200

```

14.2 小范围超参数搜索

```

hidden_dim: {64, 128}
dropout: {0.1, 0.2, 0.3}
lr: {1e-4, 3e-4, 1e-3}
lambda_keep: {0.1, 0.3, 0.5}
lambda_route: {0.1, 0.3, 0.5}
teacher_topk: {5, 10}

```

全部超参数仅根据 validation NDCG@10 选择，测试集不能参与调参。

14.3 推荐代码目录

```

cer-rec/
├── configs/
│   ├── beauty.yaml
│   ├── fashion.yaml
│   ├── steam.yaml
│   ├── baselines/
│   └── ablations/
├── data/
│   ├── raw/
│   ├── processed/
│   ├── semantic_embeddings/
│   ├── negatives/
│   └── teacher_labels/
├── src/
│   ├── data/
│   ├── embeddings/
│   ├── models/
│   ├── teachers/
│   ├── trainers/
│   └── evaluation/
├── scripts/
└── tests/

```

15. 脚本执行顺序与验收标准

Step 1: 预处理

```
bash scripts/01_preprocess.sh beauty
```

- ☐ 用户、商品和交互数量与统计文件一致。
- ☐ 训练/验证/测试严格按时间，无未来泄漏。
- ☐ 验证与测试负样本固定并可复现。

Step 2: 生成语义 Embedding

```
bash scripts/02_encode_items.sh beauty
```

- ☐ 每个 item 都能映射到唯一向量。
- ☐ 无 NaN / Inf; 缺失文本有默认处理。
- ☐ item ID 映射与 Embedding 行顺序完全一致。

Step 3: 训练 CF Expert

```
bash scripts/03_train_cf.sh beauty
```

- ☐ Validation NDCG 正常上升并早停。
- ☐ 显著优于随机和 MostPop。
- ☐ 保存最佳 checkpoint 与完整 config。

Step 4: 训练 Semantic Expert

```
bash scripts/04_train_semantic.sh beauty
```

- ☐ 显著优于随机。
- ☐ 在 Tail item 或稀疏用户上至少有一部分 unique hit。
- ☐ 无全部预测热门商品的塌缩。

Step 5: 专家诊断

```
bash scripts/05_diagnose_experts.sh beauty
```

- ☐ 输出 CF-only、Semantic-only、Both、Oracle。
- ☐ 满足 Go 标准后才继续 Teacher。

Step 6: 训练粗粒度与黑盒 Gate

```
bash scripts/06_train_gates.sh beauty
```

- ☐ Static/User/Item/UserTarget/Event 全部共享同一专家。
- ☐ EventGate 至少在一个主数据集上优于粗粒度 Gate。

Step 7: 生成 Teacher

```
bash scripts/07_generate_teacher.sh beauty
```

- ☐ Δ_{cf} 与 Δ_{sem} 不全部接近 0。
- ☐ Route target 不全部接近 0.5。
- ☐ Keep target 不全部等于 1。

Step 8: 训练完整模型

```
bash scripts/08_train_router.sh beauty
```

- ☐ Ours 稳定超过 StaticFusion。
- ☐ Teacher 相比无 Teacher 的 EventGate 有提升或明显稳健性收益。

Step 9: 补外部 Baseline

```
bash scripts/09_train_baselines.sh beauty
```

- ☐ 优先 LLM-ESR、HSUGA、RLMRec、LLMInit。
- ☐ 全部使用统一测试候选与评测代码。

Step 10: 消融与噪声实验

```
bash scripts/10_run_ablations.sh beauty
```

- ☐ 四个核心消融全部完成。
- ☐ 随机噪声历史下 Ours 性能下降更小。

16. 实验记录规范与六周计划

16.1 每次运行必须保存

- dataset、model、seed、完整 config。
- Git commit hash 与数据 checksum。
- 最佳 epoch、validation HR/NDCG、test HR/NDCG。
- 训练时间、Teacher 生成时间、GPU 峰值显存。
- 模型参数量、可训练参数量和语义 Embedding 存储大小。

```
beauty_eventgate_teacher_seed2026
steam_itemgate_seed2026
fashion_hsuga_seed2026
```

16.2 六周计划

周次	主要任务	交付物 / 决策点
第 1 周	Beauty 数据、Qwen Embedding、SASRec 与统一评测	基础数据闭环
第 2 周	CF Expert、Semantic Expert、Unique-hit 与 Oracle	Go / No-Go
第 3 周	UserGate、ItemGate、UserTargetGate、EventGate	验证事件粒度
第 4 周	Mask Teacher、Keep/Route Gate、完整模型	核心方法结果
第 5 周	Steam、Fashion、LLM-ESR、HSUGA、RLMRec	多数据集主表
第 6 周	四个消融、Noise Test、案例图、论文初稿	投稿材料

17. 论文最终结构与立即行动清单

17.1 论文贡献草稿

5. 识别并量化同一用户历史内部语义证据与协同证据的事件级异质可靠性。
6. 提出目标感知的事件级 Keep 与 Route 框架，分别判断历史是否保留以及应信任何种证据。
7. 使用独立专家的反事实 Mask 边际贡献构造事件级可靠性 Teacher，避免 Gate 退化为不可解释的注意力权重。

8. 在多个公开数据集上验证推荐性能、事件路由粒度优势、专家互补性和抗噪声能力。

17.2 论文段落结构

章节	重点
Introduction	粗粒度融合的缺陷；事件级异质性；贡献
Related Work	LLMRec、语义-协同融合、长尾/用户级方法、反事实归因
Problem Formulation	序列推荐、双专家、Keep 与 Route 定义
Method	两个专家、事件级 Gate、Teacher、训练目标
Experiments	数据、Baseline、主结果、Gate 粒度、消融
Analysis	Unique-hit / Oracle、Noise Test、案例图
Conclusion	事件级可靠性路由的结论与限制

17.3 现在立刻开始的九件事

9. 找到 HSUGA 或 LLM-ESR 的数据处理与评测代码。
10. 只先跑通 Amazon Beauty。
11. 固定验证与测试负样本。
12. 生成 Beauty 商品文本。
13. 离线生成 Qwen 语义 Embedding。
14. 跑通 SASRec 基线。
15. 实现并训练 CF Expert。
16. 实现并训练 Semantic Expert。
17. 输出 Unique-hit 与 Oracle，决定是否继续 Teacher。

第一里程碑：在 Unique-hit 和 Oracle 结果出来之前，不实现 Counterfactual Teacher。项目首先要证明的不是复杂模型能够运行，而是两个专家确实在不同样本和事件上各有独特贡献。

附录 A：最小交付检查表

- ☐ Beauty、Steam、Fashion 三个数据集使用一致的时间切分与负样本协议。
- ☐ 商品文本无未来信息泄漏。
- ☐ Qwen Embedding 离线缓存且训练期间冻结。
- ☐ CF 与 Semantic 专家独立训练、参数不共享。
- ☐ Unique-hit 与 Oracle 证明两个专家存在互补性。
- ☐ 受控 Gate baseline 只改变路由粒度，其他模块完全一致。
- ☐ 反事实 Teacher 仅由训练样本生成。
- ☐ 主结果包含 SASRec、LLMInit、RLMRec、LLM-ESR、HSUGA、Ours。
- ☐ 四个核心消融全部完成。
- ☐ 完成 Random Noise History Test。
- ☐ 关键结果至少 3 个 seed，并保存 mean $\pm$ std。
- ☐ 论文中明确区分事件相关性、事件保留和信号可靠性。

附录 B：常见失败与排查

现象	可能原因	优先排查
Semantic-only 几乎无命中	文本质量差、Embedding 弱、语义专家	检查文本字段、Embedding 对齐、Tail

现象	可能原因	优先排查
	未收敛	分桶
两个专家预测高度一致	分支信息泄漏或共享参数过多	确认 ID 与语义分支独立
EventGate 全部趋近 1	CF 先收敛导致 Gate collapse	先冻结专家；检查分支损失和初始化
Teacher 全部接近 0.5	两专家贡献无差异或 Margin 定义不敏感	检查负样本、Top-K 事件、温度
Keep 全部为 1	事件贡献尺度过大或阈值过低	标准化 Δ ；重新设 τ_k 与 T_k
Ours 只比 EventGate 多一点	Teacher 噪声大或事件粒度已捕获大部分收益	检查 Route AUROC 与高置信样本
噪声实验无优势	Keep Gate 未学会过滤或噪声太简单	可视化 q_j ；增加噪声数量

--- 文档结束 ---